UNIVERSITY OF VICTORIA

Department of Electrical and Computer Engineering

ECE 455 - Real Time Computer Systems

Project 2: Deadline-Driven Scheduler

Date Submitted: April 3, 2025

Arfaz Hossain, V00984826
Zachary Aldus, V00969072

Table of Contents

1. Introduction **3**

2. Design **4**

2.1 System Overview 4

2.2 Tasks Priority Levels 5

2.3 Design Decisions and Justifications 6

2.3.1 Single Generator vs. Per-Task Generators 6

2.3.2 Return Lists by Reference vs. by Value 6

2.3.3 Pre-create Task Handles vs. Create/Delete per Period 6

2.4 Differences from Original Design 6

3. Software Design and Implementation **7**

3.1 Data Structures 7

3.1.1 dd_task struct 8

3.1.2 dd_task_list 8

3.1.3 dds_message 8

3.1.4 dds_reply 8

3.2 Linked List Function 9

3.2.1 insert_node 9

3.3 DDS Functions

3.3.1 adjust_priorities 11

3.3.3 handle_complete 12

3.3.4 handle_deadline_miss 12

3.4 DD Scheduler Main Loop 13

3.5.1 release_dd_task 14

3.5.2 complete_dd_task 15

3.5.3 get_active_dd_task_list, get_completed_dd_task_list, and get_overdue_dd_task_list 15

3.6 Task Generator 17

3.7 User-Defined Tasks 18

3.8 Monitor Task 18

3.9 Initialization (main) 18

4. Test Results **19**

4.1 Test Bench #1 (DDS Event Log) 19

4.1.1 TB2 Trace 20

4.1.2 TB2 Discussion 20

4.2 Test Bench #2 (Monitor Task Output) 21

4.2.1 TB2 Trace 22

4.2.2 TB2 Discussion 22

4.2.3 TB2 Behaviour 22

4.3 Test Bench #3 (DDS Event Log and Observations) 23

4.3.1 TB3 Trace 24

4.3.2 TB3 Discussion 24

4.3.3 Utilization Limits 24

5. Discussion **25**
5.1 Scheduler Performance 25
5.2 Missed Deadlines 25
5.3 Discussion 26
8 - Appendix **29**
8.1 Source Code 29
8.2 Test Bench #1 Console Output 44
8.3 Test Bench #2 Console Output 45
8.4 Test Bench #3 Console Output 46

1. Introduction

A Deadline-Driven Scheduler (DDS) is a real-time scheduling system that does not use fixed task priorities. Instead, it dynamically assigns priorities based on which task has the closest deadline. This type of scheduling is important in real-time systems because missing a deadline can cause the system to fail.

The scheduling algorithm used here is Earliest Deadline First (EDF). Whenever a task is ready to run, the scheduler checks all currently active tasks and picks the one with the soonest absolute deadline. EDF is optimal for single-processor systems, meaning it can schedule any task set that any other algorithm can, as long as the total CPU utilization stays at or below 100%. Because priorities are assigned based on deadlines rather than being fixed ahead of time, EDF is more flexible than fixed-priority approaches like Rate Monotonic Scheduling.

Since FreeRTOS is not configurable enough to natively perform deadline-driven scheduling, the DDS has to be built on top of the existing FreeRTOS task scheduler. To keep things clear, a task managed by the DDS is called a Deadline-Driven Task (DD-Task), while a task managed by FreeRTOS is called a FreeRTOS Task (F-Task). A DD-Task is not a real task but is simply a data structure that holds the handle of a corresponding user-defined F-Task along with the information needed for EDF scheduling.

The goal of this project is to implement a custom DDS using EDF scheduling on the STM32F4 Discovery Board, running on top of FreeRTOS. The system was tested against three test benches with different utilization levels to verify correct behavior.

2. Design

2.1 System Overview

The DDS scheduler schedules tasks by giving the highest priority to the task with the nearest absolute deadline, and assigning the lowest priority to all other tasks. It sorts the Active List by the earliest deadline, moves finished tasks to the Completed List, moves tasks missing deadlines to the Overdue List, and informs the generator if a task needs rescheduling.

The Generator Task releases tasks to DDS, triggered by a software timer set to each task's period. When triggered, the generator wakes, selects a task, and sends its release message to DDS, including the task ID.

Once scheduled by DDS, the User Task runs a busy loop to represent execution. After completion, it notifies DDS, which moves it from the active to the complete list and schedules the next task.

The Monitor Task requests and prints the Active, Completed, and Overdue lists, along with the task count, from DDS. The DDS queue receives messages from the generator, user tasks, and monitor. Release requests, completion messages, and list requests are processed. If a task-complete message is not received before the head task's deadline, the queue timeout signals a missed deadline, moving that task from the active list to the Overdue List.

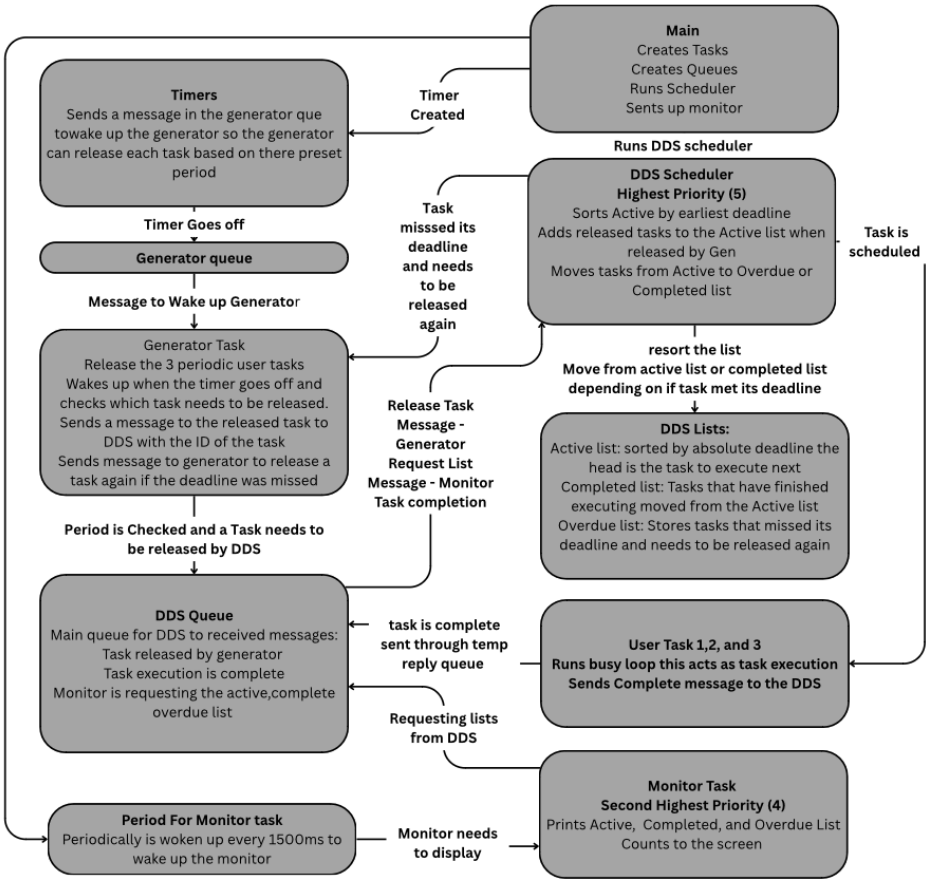

Figure 1: System Overview Diagram

2.2 Tasks Priority Levels

See the Below Table for the Purpose of each F-Task and our justification for the given priority level:

Table 1: F-Task Priority

F-Task	Priority	Purpose	Priority Justification
DD_Scheduler_Task	5 (Highest)	Handles scheduling tasks it is the control unit using EDF task scheduling. Additionally it sorts the active list by the earliest deadline, moves tasks from the active to completed or overdue (missed) list. And receives messages from the generator for a release monitor for a list request to print and user tasks for completion of the busy loop	The Scheduler needs to be the highest priority because it must reorder the active list whenever a new task is released through a message from the generator. Additionally it must preempt all other tasks because it is the controller and decides which task needs to be executed based on an absolute deadline.
Monitor_Task	4	Periodically requests list from the DDS and displays the count to the screen	We originally planned to have the monitor run at a lower priority like the the lab manual suggests but we found that the monitor would not get the cpu time needed to print to the display because it would be preempted by the current head of the active list that the DDS has scheduled and set to a high priority.
DD_Generator_Task	3	Utilizes a timer to release tasks through the DDS queue for the DDS to add to the active list. Additionally calculates deadlines and calls release_dd_task.	Higher than user tasks because releasing one or multiple tasks to the DDS is important to ensure that the task executes at for example 0ms rather than induce a drift that eats CPU time and under certain tests will cause the system to fail.
User_Task 1–3	2 (HIGH) or 1 (LOW)	Runs busy loop to represent executing the task	The DDS gives the head of the list highest priority when its scheduled and all other task receive the lowest priority this ensures that the most urgent task with the closest absolute deadline is executed

2.3 Design Decisions and Justifications

2.3.1 Single Generator vs. Per-Task Generators

Single generator you have one generator task that takes care of all tasks using timers to release tasks. Having a single generator allows us to have less overhead then if one per task each task had its own queue and messages that would likely lead to more conflicts

2.3.2 Return Lists by Reference vs. by Value

Return by Pointer to avoid having to copy over the list when returned which with a longer list can take a lot of time

2.3.3 Pre-create Task Handles vs. Create/Delete per Period

Static tasks using xTaskCreate(). We are going to initialize our tasks on startup and, whenever they need to run, wake them up with vTaskResume(), and once they are complete, put them back to sleep with vTaskSuspend(NULL). We did not choose to create and delete a task every time because each time we create a task, it takes memory and creates a stack with its own task control block, which also takes Memory on the STM discovery. We did think about deleting the task, but even if we did, we risk fragmenting the memory, which is where we might have enough, but it's not in a continuous block; it's broken up into pieces.

2.4 Differences from Original Design

The final implementation is close to the original design, but a few things changed. The biggest change is how deadline misses are detected. Instead of using a separate software timer, the DDS uses the timeout on xQueueReceive. The timeout is set to the time remaining until the head task's deadline is hit, and if it expires without a message arriving, the deadline is missed. This turned out to be simpler than managing separate timers. Another change is that each DD-Task instance gets a unique instance_id in addition to the task_id. The task_id identifies which user task it is Task 1, 2, or 3, but since the same task is released multiple times across different periods, the instance_id lets the DDS distinguish between different releases of the same task.

3. Software Design and Implementation

3.1 Data Structures

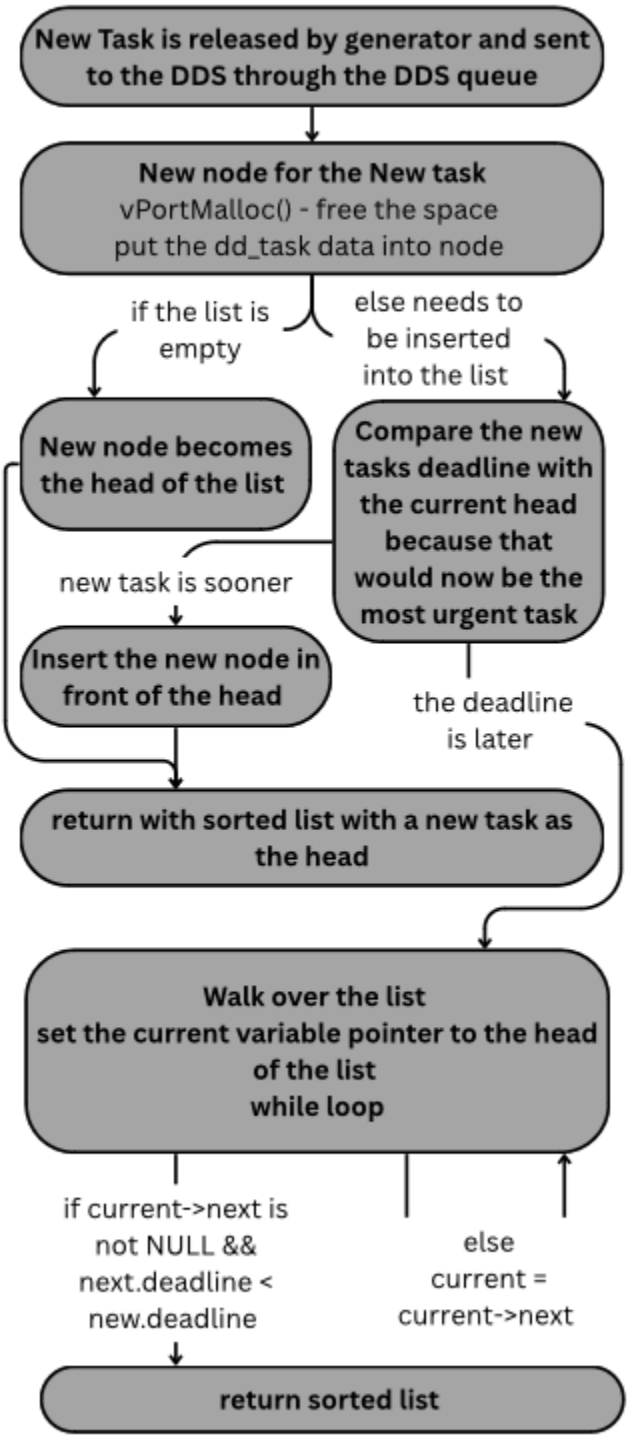

Figure 2: Linked List Flow Chart

3.1.1 dd_task struct

This struct stores the scheduling information for one task instance. The fields are: t_handle is the FreeRTOS task handle for the corresponding user-defined F-Task, type is either PERIODIC or APERIODIC, task_id identifies which user task this belongs to 1, 2, or 3, instance_id is a unique number assigned each time a task is released so different instances of the same periodic task can be told apart, release_time is the tick count when the task was added to the active list, absolute_deadline is the hard deadline in ticks, and completion_time is the tick count when the task finished executing.

```
typedef struct {
   TaskHandle_t t_handle;
   task_type type;
   uint32_t task_id;
   uint32_t instance_id;
   uint32_t release_time;
   uint32_t absolute_deadline;
   uint32_t completion_time;
} dd_task;
```

3.1.2 dd_task_list

A singly-linked list node. Each node embeds a dd_task and a pointer to the next node, forming the sorted active list, the completed list, and the overdue list.

```
typedef struct dd_task_node {
   dd_task task;
   struct dd_task_node *next;
} dd_task_list;
```

3.1.3 dds_message

The message passed through the DDS queue. Contains a msg_type field (MSG_RELEASE, MSG_COMPLETE, MSG_GET_ACTIVE, MSG_GET_COMPLETED, MSG_GET_OVERDUE), the dd_task payload, and an optional reply_queue handle for request-response queries.

```
typedef struct {
   uint32_t msg_type;
   dd_task task;
   QueueHandle_t reply_queue;
} dds_message;
```

3.1.4 dds_reply

A simple wrapper containing a pointer to a dd_task_list, sent back through the reply queue in response to list queries.

```
typedef struct { dd_task_list *list; } dds_reply;
```

3.2 Linked List Function

Three helper functions are used to manage the DD-Task linked lists:

3.2.1 insert_node

This function allocates a new dd_task_list node with pvPortMalloc, copies the dd_task data into it, and inserts it into the list sorted by absolute_deadline with the earliest deadline at the head.

```
void insert_node(dd_task_list **head, dd_task task)
{
   dd_task_list *new_node;
   dd_task_list *curr;
   new_node = (dd_task_list
*)pvPortMalloc(sizeof(dd_task_list)); // alloc mem  for new
node
   if (new_node == NULL) { return; }
   new_node->task = task;
   new_node->next = NULL;

   if (*head == NULL) { *head = new_node; return; }
   // insert at front if earlier deadline  ,EDF priority
basically
   if (task.absolute_deadline <
(*head)->task.absolute_deadline)
   {
       new_node->next = *head;
       *head = new_node;
       return;
   }

   curr = *head;
   // looping till correct spot  keeping list sorted by
deadline
   while (curr->next != NULL &&
          curr->next->task.absolute_deadline <=
task.absolute_deadline)
       curr = curr->next;

   new_node->next = curr->next;
   curr->next = new_node;}
```

3.2.2 remove_node

This function searches the list for a node that matches the given instance_id. If it finds a match, it unlinks the node from the list and returns a pointer to it. It handles the case where the node to

remove is the head as well as when it is in the middle of the list. If no matching instance_id is found, it returns NULL.

```
dd_task_list *remove_node(dd_task_list **head, uint32_t
instance_id)
{
   dd_task_list *prev, *curr;
   if (*head == NULL) return NULL;
   // special case: removing head node  gotta update head pointer
   if ((*head)->task.instance_id == instance_id)
   {
       dd_task_list *r = *head;
       *head = (*head)->next;
       r->next = NULL;
       return r;
   }

   prev = *head;
   curr = (*head)->next;
   while (curr != NULL) {
       // traverse list to find matching instance id
       if (curr->task.instance_id == instance_id)
       {
           prev->next = curr->next;
           curr->next = NULL;
           return curr;
       }
       prev = curr;
       curr = curr->next;
   }
   return NULL;
}
```

3.2.3 get_list_size

This function walks through the list, counts the number of nodes, and returns the count. It is mainly used by the Monitor Task to report how many tasks are in each list.

```
uint32_t get_list_size(dd_task_list *head)
{
   uint32_t n = 0;
   while (head != NULL) { n++; head = head->next; }
   return n;
}
```

3.3 DDS Functions

3.3.1 adjust_priorities

Goes through the active list and sets the head node's F-Task to PRIORITY_USER_HIGH (2) and all other F-Tasks to PRIORITY_USER_LOW (1). This way, FreeRTOS will only run the user task that has the earliest deadline, since it is the only one with a high enough priority to execute.

```
void adjust_priorities(void)
{
   dd_task_list *curr = active_list;
   while (curr != NULL) {
       // updating priority dynamically based on EDF
       vTaskPrioritySet(curr->task.t_handle,
                        (curr == active_list) ?
PRIORITY_USER_HIGH
                                              :
PRIORITY_USER_LOW);
       curr = curr->next;
   }
}
```

3.3.2 handle_release

This gets called when a MSG_RELEASE message arrives. It records the current tick count as the release_time, adds the current tick count to the relative deadline to get the absolute_deadline, inserts the DD-Task into the active list using insert_node (which keeps it sorted), calls adjust_priorities to update which task should run, and prints the release event.

```
void handle_release(dds_message *msg)
{
   TickType_t now = xTaskGetTickCount();
   msg->task.release_time = (uint32_t)now;
   msg->task.absolute_deadline = (uint32_t)(now +
(TickType_t)msg->task.absolute_deadline);
   insert_node(&active_list, msg->task);
   adjust_priorities();
   sem_printf("T%u R:%ums\n",
          (unsigned int)msg->task.task_id,
          (unsigned int)TICKS_TO_MS(msg->task.release_time));
}
```

3.3.3 handle_complete

This gets called when a MSG_COMPLETE message arrives. It removes the matching DD-Task from the active list using remove_node (matching by instance_id), records the current tick count as the completion_time, sets the completed task's priority to LOW, adds the node to the front of the completed list, calls adjust_priorities, and prints the completion event.

```
void handle_complete(dds_message *msg)
{
   // take task out of active list when done
   dd_task_list *node = remove_node(&active_list,
msg->task.instance_id);
   if (node != NULL)
   {
       node->task.completion_time = xTaskGetTickCount();
       vTaskPrioritySet(node->task.t_handle,
PRIORITY_USER_LOW);
       node->next = completed_list; // push to completed
list, not sorted btw just stack
       completed_list = node;
       adjust_priorities();
       sem_printf("T%u C:%ums\n",
              (unsigned int)node->task.task_id,
              (unsigned
int)TICKS_TO_MS(node->task.completion_time));
   } else {
       adjust_priorities();
   }
}
```

3.3.4 handle_deadline_miss

This gets called when xQueueReceive times out, meaning no message arrived before the head task's deadline expired. It takes the head of the active list (the task that missed its deadline), sets its F-Task priority to LOW, adds it to the front of the overdue list, calls adjust_priorities, and prints the miss event.

```
void handle_deadline_miss(void)
{
   dd_task_list *missed;
   if (active_list == NULL) { return; }
   missed = active_list; // take the earliest deadline task
head as missed
   active_list = active_list->next;
   missed->next = NULL;
   vTaskPrioritySet(missed->task.t_handle,
PRIORITY_USER_LOW);
   missed->next = overdue_list;
```

```
    overdue_list = missed; // move it to overdue list
    adjust_priorities();
    sem_printf("T%u M:%ums\n",
           (unsigned int)missed->task.task_id,
           (unsigned int)TICKS_TO_MS(xTaskGetTickCount()));}
```

3.4 DD Scheduler Main Loop

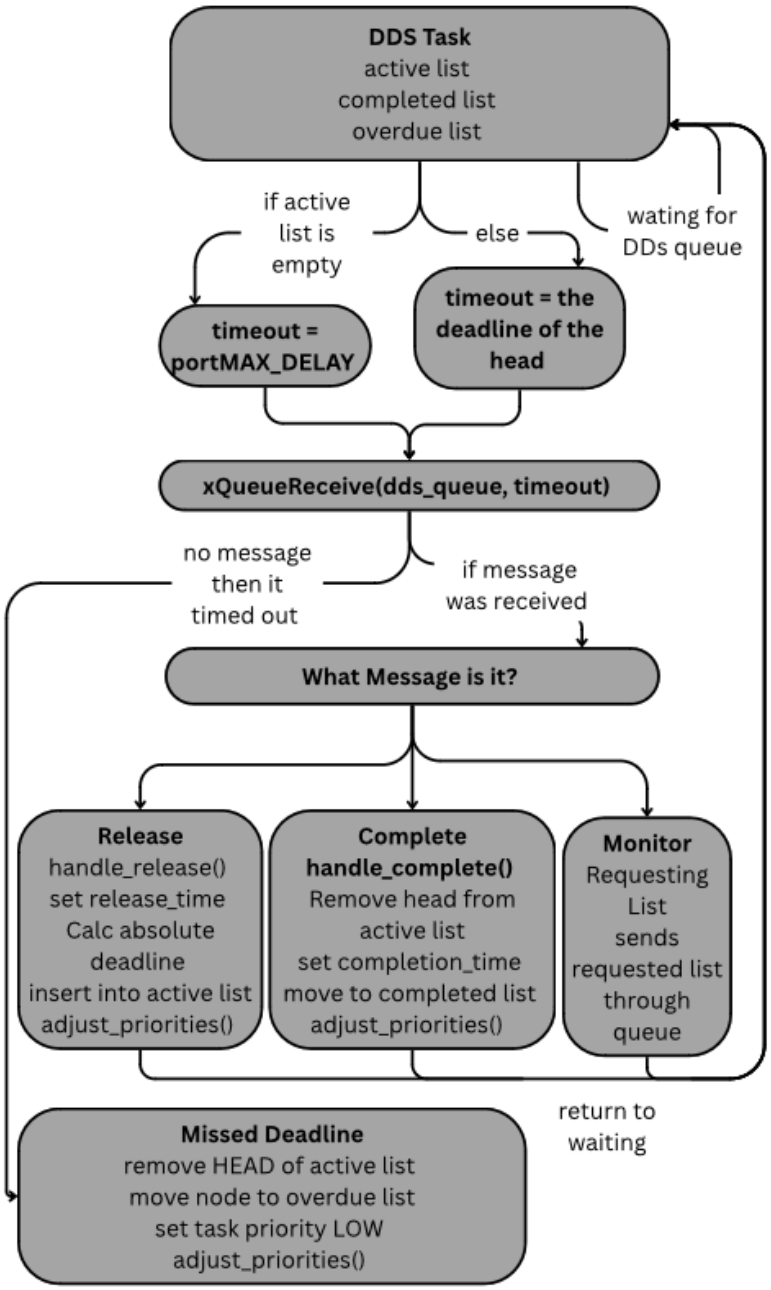

Figure 3: Scheduler Flow Chart

The DD Scheduler Task is blocked while waiting for a message from the DDS queue or a timeout occurs. It calculates a timeout using get_next_timeout, which returns the time remaining until the head task's absolute deadline. If there are no active tasks, the timeout is set to portMAX_DELAY so the scheduler just waits until a message arrives.

The task then calls xQueueReceive on dds_queue with this timeout. If a message arrives (pdTRUE), the scheduler checks the message type: MSG_RELEASE goes to handle_release, MSG_COMPLETE goes to handle_complete, and the three list query types (MSG_GET_ACTIVE, MSG_GET_COMPLETED, MSG_GET_OVERDUE) send back the corresponding list pointer through the reply queue included in the message.

If xQueueReceive returns pdFALSE, that means the timeout expired without any message, so the head task's deadline has been missed and handle_deadline_miss is called. This way the queue timeout also acts as the deadline watchdog, so a separate timer for detecting misses is not needed.

3.5 DDS Interface Functions

3.5.1 release_dd_task

Takes a task handle, task type, task ID, and relative deadline. It assigns a unique instance_id using an auto-incrementing counter, puts all the information into a dds_message with msg_type set to MSG_RELEASE, and sends it to dds_queue. It returns the instance_id so the user task knows which instance it is running.

```
uint32_t release_dd_task(TaskHandle_t h, task_type t,
uint32_t id,
                    uint32_t relative_deadline)
{
   dds_message msg;
   uint32_t iid = next_instance_id++; // unique id for each
task instance, auto increment
   msg.msg_type = MSG_RELEASE;
   msg.task.t_handle = h;
   msg.task.type = t;
   msg.task.task_id = id;
   msg.task.instance_id = iid;
   msg.task.release_time = 0;
   msg.task.absolute_deadline = relative_deadline;
   msg.task.completion_time = 0;
   msg.reply_queue = NULL;
   // send msg to scheduler, blocking if needed
   xQueueSend(dds_queue, &msg, portMAX_DELAY);
   return iid;
}
```

3.5.2 complete_dd_task

This takes an instance_id, packages it into a dds_message with msg_type set to MSG_COMPLETE, and sends it to dds_queue. No reply is needed for this one.

```
void complete_dd_task(uint32_t instance_id)
{
   dds_message msg;
   msg.msg_type = MSG_COMPLETE;
   msg.task.instance_id = instance_id;
   msg.reply_queue = NULL;
   xQueueSend(dds_queue, &msg, portMAX_DELAY);
}
```

3.5.3 get_active_dd_task_list, get_completed_dd_task_list, and get_overdue_dd_task_list

Each of these creates a temporary queue that can hold one reply, sends a query message to dds_queue with the reply queue handle attached, blocks on the reply queue waiting for the DDS to send back the list pointer, deletes the temporary queue, and returns the pointer. Using a temporary reply queue for each request makes sure that access to the internal lists is thread-safe without needing any extra synchronization.

```
dd_task_list *get_active_dd_task_list(void)
{
   // temp queue for reply
   QueueHandle_t rq = xQueueCreate(1, sizeof(dds_reply));
   dds_message msg; dds_reply reply;
   msg.msg_type = MSG_GET_ACTIVE; msg.reply_queue = rq;
   xQueueSend(dds_queue, &msg, portMAX_DELAY);
   xQueueReceive(rq, &reply, portMAX_DELAY);
   vQueueDelete(rq);
   return reply.list;
}
```

```
dd_task_list *get_completed_dd_task_list(void)
{
   QueueHandle_t rq = xQueueCreate(1, sizeof(dds_reply));
   dds_message msg; dds_reply reply;
   msg.msg_type = MSG_GET_COMPLETED; msg.reply_queue = rq;
   xQueueSend(dds_queue, &msg, portMAX_DELAY);
   xQueueReceive(rq, &reply, portMAX_DELAY); // wait for
scheduler response
```

```
    vQueueDelete(rq);
    return reply.list;
}
```

```
dd_task_list *get_overdue_dd_task_list(void)
{
    QueueHandle_t rq = xQueueCreate(1, sizeof(dds_reply));
    dds_message msg; dds_reply reply;
    msg.msg_type = MSG_GET_OVERDUE; msg.reply_queue = rq;
    xQueueSend(dds_queue, &msg, portMAX_DELAY);
    xQueueReceive(rq, &reply, portMAX_DELAY);
    vQueueDelete(rq);
    return reply.list;
}
```

3.6 Task Generator

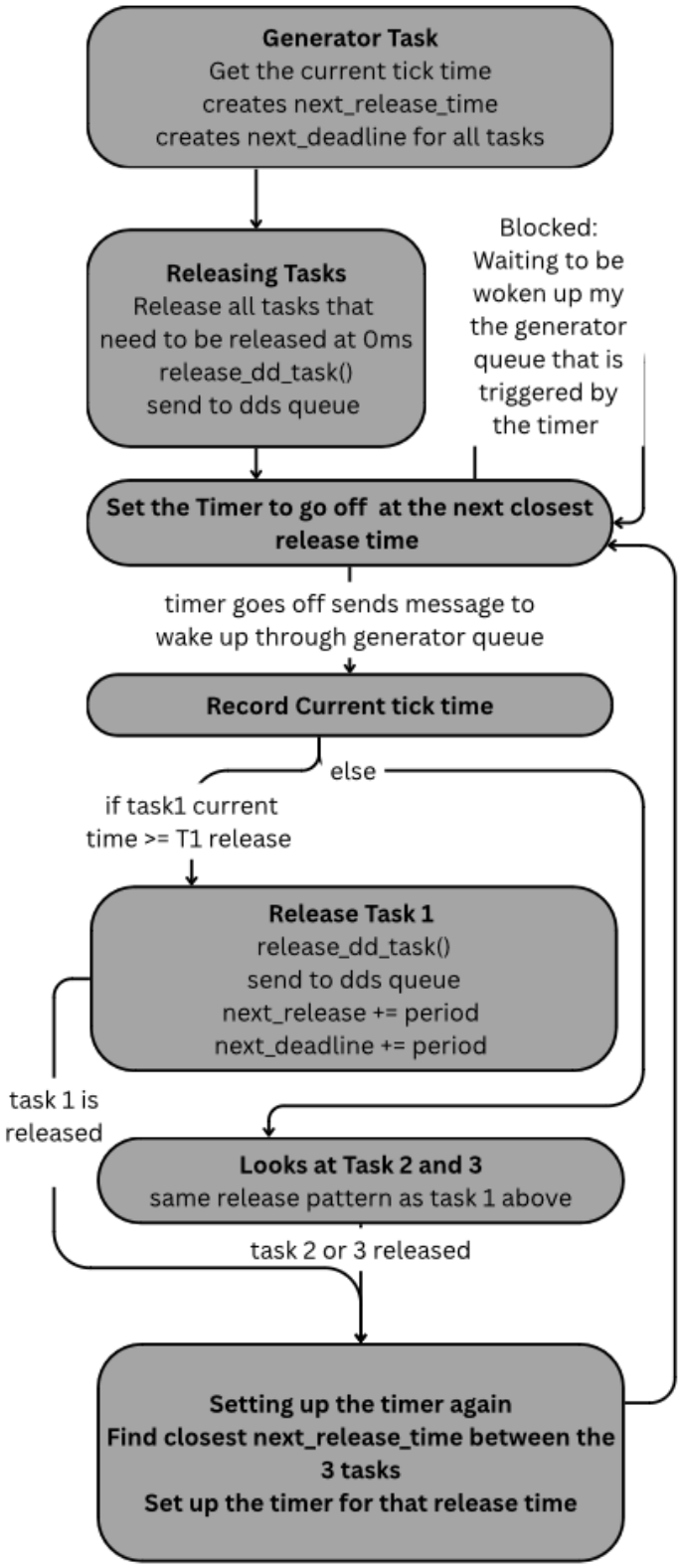

Figure 4: Task Generator Flow Chart

The Generator Task handles periodic task releases using a single one-shot software timer called release_timer that gets re-armed after each set of releases.

When the Generator starts up, it creates the three per-task release queues (task1_release_queue, task2_release_queue, task3_release_queue) and the three User-Defined F-Tasks at LOW priority. Since the user tasks start at LOW priority, they immediately block on their release queues waiting for work. The Generator records the initial tick count as t0, sets up the initial release and deadline ticks for each task, does the initial releases at t = 0 by calling release_due_tasks, and arms the timer for the next release event.

The release timer callback just sends a GENERATOR_WAKE_UP token to generator_queue. The Generator Task's main loop blocks on this queue. When it gets a token, it calls release_due_tasks, which checks each task's next_release_tick against the current tick count and releases any tasks that are due. For each released task, the next_release_tick and next_deadline_tick are advanced by the task's period. After releasing everything that is due, the timer is re-armed for the next earliest release.

Each task tracks its own next_deadline_tick which advances by its period on every release. This keeps the deadlines from drifting over time even if there is some jitter in when the releases actually happen.

3.7 User-Defined Tasks

The task blocks the release queue. When it gets one, it runs a busy-wait loop for its configured execution time using busy_wait_ms, which uses the ARM DWT cycle counter to get timing precision below the 1 ms tick resolution. When the busy-wait finishes, the task calls complete_dd_task with its instance_id to let the DDS know it is done, and then loops back to block on the release queue again. As required by the lab manual, the user tasks are fully self-contained and do not communicate with other tasks during execution.

3.8 Monitor Task

The Monitor Task runs at priority 4. We did not run it at the recommended low priority because we observed it would not get the dedicated CPU time to actually print the lists. Its period was set to 1500 ms, the end of a hyper period. It calls get_active_dd_task_list, get_completed_dd_task_list, and get_overdue_dd_task_list, counts the nodes in each using get_list_size, and prints the numbers using sem_printf. We used a mutex to prevent multiple tasks from printing at the same time.

3.9 Initialization (main)

The main function sets everything up before starting the scheduler. It initializes the DWT cycle counter for busy-wait timing, creates dds_queue with a capacity of 16 messages and generator_queue with a capacity of 8 tokens, and creates the print mutex. Then it creates the three main F-Tasks: DD_Scheduler_Task at priority 5, Monitor_Task at priority 4, and DD_Generator_Task at priority 3. After that it calls vTaskStartScheduler to start FreeRTOS. Once the scheduler is running, the Generator Task takes care of the rest of the setup: creating the user tasks, setting up the release queues, configuring the release timer, and doing the initial releases at t = 0.

4. Test Results

4.1 Test Bench #1 (DDS Event Log)

Task 1 has an execution time of 95 ms and a period of 500 ms. Task 2 has an execution time of 150 ms and a period of 500 ms. and Task 3 has an execution time of 250 ms and a period of 750 ms.

The total utilization is U = 95/500 + 150/500 + 250/750 = 0.190 + 0.300 + 0.333 = 0.823.
This means that all deadlines should be met because we did not use up all the cpu time and the utilization is less then 1.

Table 4: DDS events for Test Bench #1

#	Event	Expected (ms)	Measured (ms)
1	Task 1 Released	0	0
2	Task 2 Released	0	0
3	Task 3 Released	0	0
4	Task 1 Complete	95	95
5	Task 2 Complete	245	246
6	Task 3 Complete	495	496
7	Task 1 Released	500	500
8	Task 1 Complete	595	595
9	Task 2 Released	500	500
10	Task 2 Complete	745	745
11	Task 3 Released	750	750
12	Task 1 Released	1000	1000
13	Task 1 Complete	1095	1096
14	Task 3 Complete	1245	1001
15	Task 2 Released	1000	1000
16	Task 2 Complete	1395	1246
17	Task 1 Released	1500	1500

The Monitor Task output at 1500 ms reported: M:a-0 c-8 o-0, this is what we expected because the utilization was not that close to 100%

4.1.1 TB2 Trace

At t = 0, all three tasks are released simultaneously. Task 1 and Task 2 both have deadlines at 500 ms, and Task 3 has a deadline at 750 ms. EDF picks Task 1 first (ties are broken by insertion order), and it runs from 0 to 95 ms. Task 2 runs next from 95 to 246 ms, then Task 3 from 246 to 496 ms.

At t = 500, Tasks 1 and 2 are released again, with deadlines of 1000 ms. No other tasks are active at this point, so Task 1 runs from 500 to 595 ms and Task 2 runs from 595 to 745 ms. Task 3 gets released at 750 ms with a deadline of 1500 ms and starts right away since nothing else is active.

At t = 1000, Tasks 1 and 2 are released again, each with a deadline of 1500 ms. By this time, Task 3 has been running for 750 ms and has already used up its full 250 ms of execution time. The DDS briefly preempts Task 3 to process the release messages, and Task 3's completion gets recorded at 1001 ms. Task 1 runs from 1001 to 1096 ms, and Task 2 runs from 1096 to 1246 ms. All tasks met their deadlines.

4.1.2 TB2 Discussion

The measured times are very close to the expected values, with all differences being 1 ms or less. These small differences are due to the overhead of processing printing to the screen; this can take a lot of CPU time even when reducing the amount you are printing. There is also a small drift caused by initialization which gives the initial tasks enough time to be released.

The lab manual's expected table has Task 3 completing at 1245 ms. That assumes Task 3 gets preempted at t = 1000 when Task 1 and Task 2 get released and has to wait for both of them to finish. What actually happened is Task 3 finished at 1001 ms. It had been running since 750 ms so by t = 1000 it already ran for 250 ms which is its full execution time. The DDS just needed about 1 ms to process the release messages for Task 1 and Task 2 before it could record Task 3 as complete, since the DDS runs at a higher priority than the user tasks.

EDF made the right call at every point and all deadlines were met. The reason it looks different from the expected table is just how the system handles ties. If a task is already running and a new task comes in with the same deadline, the running task keeps going. That is a perfectly fine way to handle ties in EDF.

4.2 Test Bench #2 (Monitor Task Output)

Task 1 has execution time 95 ms and period 250 ms, Task 2 has execution time 150 ms and period 500 ms, and Task 3 has execution time 250 ms and period 750 ms.

The utilization is U = 95/250 + 150/500 + 250/750 = 0.380 + 0.300 + 0.333 = 1.013.

Utilization is above 100% so we expect there to be missed at the end of a hyper period the

Table 5: Monitor Task output for Test Bench #2 after one hyper-period (1500 ms)

Metric	Expected	Measured
Number of active DD-Tasks	0	0
Number of completed DD-Tasks	-	10
Number of overdue DD-Tasks	≥ 1	1

Table 6: DDS events for Test Bench #2 (full hyper-period)

#	Event	Expected (ms)	Measured (ms)	Difference (ms)
1	Task 1 Released	0	0	0
2	Task 2 Released	0	0	0
3	Task 3 Released	0	0	0
4	Task 1 Complete	95	95	0
5	Task 2 Complete	245	246	+1
6	Task 1 Released	250	250	0
7	Task 1 Complete	345	345	0
8	Task 1 Released	500	500	0
9	Task 2 Released	500	500	0
10	Task 3 Complete	591	592	+1
11	Task 1 Complete	686	687	+1
12	Task 1 Released	750	750	0
13	Task 3 Released	750	750	0
14	Task 2 Complete	837	838	+1
15	Task 1 Complete	932	933	+1
16	Task 1 Released	1000	1000	0
17	Task 2 Released	1000	1000	0
18	Task 1 Complete	1095	1095	0
19	Task 1 Released	1250	1250	0
20	Task 3 Complete	1278	1279	+1
21	Task 2 Complete	1428	1430	+2
22	Task 1 Deadline Miss	1500	1500	0

The Monitor Task output at 1500 ms reported: M:a-0 c-10 o-1.

4.2.1 TB2 Trace

At t = 0, all three tasks are released. Task 1 has the earliest deadline (250 ms), so it runs first and completes at 95 ms. Task 2 runs next (deadline 500 ms), completing at 246 ms. Task 3 starts at 246 ms with a deadline of 750 ms, but at t = 250, Task 1 gets released again with a deadline of 500 ms. Since 500 < 750, Task 1 preempts Task 3. At this point, Task 3 has only run for about 4 ms.

Task 1 completes at 345 ms, and Task 3 resumes. Task 3 keeps running until t = 500, when Task 1 (deadline 750) and Task 2 (deadline 1000) are released. Task 3's deadline is also 750, the same as Task 1's, but since Task 3 was already in the active list, it keeps running. Task 3 has about 91 ms left and finishes at 592 ms.

Task 1 then runs from 592 to 687 ms. Task 2 starts at 687 ms. At t = 750, Task 1 (deadline 1000) and Task 3 (deadline 1500) are released. Task 2's deadline (1000) ties with Task 1's, and since Task 2 was already running, it continues. Task 2 finishes at 838 ms, then Task 1 finishes at 933 ms. Task 3 starts at 933 ms.

At t = 1000, Task 1 (deadline 1250) and Task 2 (deadline 1500) are released. Task 1 preempts Task 3 because 1250 < 1500. Task 1 finishes at 1095 ms, and Task 3 resumes. At t = 1250, Task 1 is released again with deadline 1500. Task 3, Task 2, and Task 1 all share a deadline of 1500 now, so the existing order is kept. Task 3 finishes at 1279 ms, Task 2 finishes at 1430 ms.

At this point, the sixth instance of Task 1 (released at 1250, deadline 1500) has only 70 ms left before its deadline, but needs 95 ms. The DDS queue timeout fires at 1500 ms, and the deadline is missed.

4.2.2 TB2 Discussion

All measured times are within 2 ms of the expected values, matching the overhead observed in Test Bench 1. The overhead adds up a bit over the hyper-period: Task 2's completion at event 21 is 2 ms late instead of 1 ms, which makes sense since more DDS messages have been processed by then.

The 1500 ms deadline miss was expected, given the utilization of 1.013. Over the full hyper-period, the tasks need 1.013 * 1500 = 1519.5 ms of CPU time, but only 1500 ms is available. That 19.5 ms shortfall shows up as the last Task 1 instance not having enough time. It needed 95 ms of execution, but there were only 70 ms left after Task 2 finished at 1430 ms.

EDF handled the overload well. Instead of causing multiple deadline misses early on, it pushed the failure to the end of the hyper-period and completed 10 out of 11 task instances on time. Since EDF is optimal for single-processor systems, no other scheduling algorithm could have done better.

4.2.3 TB2 Behaviour

Test Bench 2 had a lot of preemptions happening. Task 3 got interrupted twice in its first instance, once at t = 250 when Task 1 came in and once at t = 500 when the DDS had to recheck priorities. Then in its second instance it got preempted again at t = 1000 by Task 1. Every time this happened the DDS left Task 3 in the active list where it was, ran the higher priority task, and then Task 3 picked back up from where it stopped. Both times Task 3 ended up running for about 250 ms total plus a bit of overhead from the DDS, so no execution time got lost from the preemptions.

4.3 Test Bench #3 (DDS Event Log and Observations)

Task 1 has execution time 100 ms and period 500 ms, Task 2 has execution time 200 ms and period 500 ms, and Task 3 has execution time 200 ms and period 500 ms. The utilization is U = 100/500 + 200/500 + 200/500 = 0.200 + 0.400 + 0.400 = 1.000. This puts the system right at the theoretical schedulability boundary.

Table 7: DDS events for Test Bench #3 (full hyper-period)

#	Event	Expected (ms)	Measured (ms)	Difference (ms)
1	Task 1 Released	0	0	0
2	Task 2 Released	0	0	0
3	Task 3 Released	0	0	0
4	Task 1 Complete	100	100	0
5	Task 2 Complete	300	301	+1
6	Task 3 Deadline Miss	500	500	0
7	Task 1 Released	500	500	0
8	Task 2 Released	500	500	0
9	Task 3 Released	500	500	0
10	Task 1 Complete	600	601	+1
11	Task 2 Complete	800	801	+1
12	Task 3 Deadline Miss	1000	1000	0
13	Task 1 Released	1000	1000	0
14	Task 2 Released	1000	1000	0
15	Task 3 Released	1000	1000	0
16	Task 1 Complete	1100	1101	+1
17	Task 2 Complete	1300	1301	+1
18	Task 3 Deadline Miss	1500	1500	0

Table 8: Monitor Task output for Test Bench #3 after one hyper-period (1500 ms)

Metric	**Measured**
Number of active DD-Tasks	0
Number of completed DD-Tasks	6
Number of overdue DD-Tasks	3

The Monitor Task output at 1500 ms: M:a-0 c-6 o-3.

4.3.1 TB3 Trace

Since all three tasks share the same period of 500 ms, the scheduling pattern repeats every period.

In the first period (0 to 500 ms), all three tasks are released at t = 0 with identical deadlines of 500 ms. Ties are broken by insertion order, so Task 1 runs first. Task 1 completes at 100 ms and Task 2 starts right away, completing at 301 ms (the extra 1 ms is DDS overhead). Task 3 starts at 301 ms. It needs 200 ms but only has 199 ms left before the 500 ms deadline. When the DDS queue timeout fires at t = 500, Task 3 gets marked as overdue and moved to the overdue list. It was short by about 1 ms.

The second period (500 to 1000 ms) is the same pattern. All three tasks are released at t = 500 with deadlines at 1000 ms. Task 1 completes at 601 ms, Task 2 at 801 ms, and Task 3 misses at 1000 ms. The third period (1000 to 1500 ms) is identical: Task 1 at 1101 ms, Task 2 at 1301 ms, Task 3 misses at 1500 ms.

4.3.2 TB3 Discussion

The 1 ms overhead on completion times (events 5, 10, 11, 16, 17) comes from the DDS processing each MSG_COMPLETE message. When Task 1 completes, the DDS has to receive the message, remove Task 1 from the active list, add it to the completed list, adjust priorities, and let Task 2 start running. All of that takes about 1 ms.

This 1 ms overhead is what causes all three deadline misses. In a system with zero overhead, Task 1 would finish at exactly 100 ms, Task 2 at exactly 300 ms, and Task 3 would have exactly 200 ms left to meet the 500 ms deadline. But with the 1 ms overhead on Task 2's completion, Task 3 does not start until 301 ms, leaving only 199 ms. Since Task 3 needs 200 ms, it misses by 1 ms every single period.

4.3.3 Utilization Limits

Test Bench 3 shows that 100% theoretical utilization is not possible to achieve on a real system. Even though the DDS only adds about 1 ms of overhead per scheduling decision, that is enough to push the effective utilization above 1.0 when context switches, queue operations, and priority adjustments are taken into account.

The deadline misses are deterministic and repeatable. Every period has the same pattern: Task 1 and Task 2 complete, Task 3 misses. This happens because the overhead is consistent and the task set has a symmetric structure. Task 3 always misses because it is the last task to run in each period. Since all three tasks have identical deadlines, the tie-breaking order (insertion order) decides which task runs last, and that task absorbs all the accumulated overhead. Despite three deadline misses, the scheduler does not have cascading failures. It resets correctly at the start of each period and keeps meeting Task 1 and Task 2 deadlines without any degradation.

If the overhead could be reduced, for example by removing the printf calls used for debugging, Task 3 might meet its deadline. Reducing any task's execution time by even 2 ms would bring the effective utilization below 1.0 and let all deadlines be met.

5. Discussion

5.1 Scheduler Performance

This project built a Deadline-Driven Scheduler using the Earliest Deadline First algorithm. We modified the FreeRTOS priority-based scheduler from its original fixed scheduling priority and ran it on the STM32F4 Discovery Board. The DDS system has five interface functions: release_dd_task, complete_dd_task, get_active_dd_task_list, get_completed_dd_task_list, and get_overdue_dd_task_list.

Our DDS system was tested on 3 different test benches, each with higher CPU utilization. Higher utilization means task periods are shorter and tasks run more frequently within the 1500ms hyperperiod. In the first test bench, utilization was below 100%, and the system ran smoothly with no missed deadlines.

This shows that EDF reliably schedules tasks with sufficient CPU time. Test bench 2 demonstrated that the monitor could request the active, completed, and overdue lists from itself and print the counts of each list periodically. The counts were displayed at the end of the 1500ms hyperperiod. We also increased utilization on this test bench to see what the system could handle as all tasks' execution time approached CPU capacity. The third test bench ran at 100% utilization, fully using the CPU. The system did not perform as expected. This was likely because the drift in execution time from printing to the screen accumulated, even though we reduced the amount printed.

The DDS system needs to keep communication overhead low since exact release and completion times become critical as utilization increases. A single generator task is used instead of one per task type to reduce overhead and avoid multiple tasks sending messages to the DDS queue at the same time. The generator uses software timers to trigger periodic releases. When the monitor requests the task lists, pointers to the internal lists are passed back instead of copying the lists. This avoids the time and memory overhead of making deep copies.

All user tasks are created once during initialization rather than being created and deleted every period. This prevents memory fragmentation from repeated calls to xTaskCreate and vTaskDelete. Even though the test benches only run three tasks for one hyper-period, pre-creating the task handles is a safer approach if more tasks were added or if the system needed to run for longer periods.

Our DDS performed well under low and almost 100% utilization. It schedules tasks based on the closest absolute deadline. Communication between tasks remained reliable through queues.

5.2 Missed Deadlines

Deadline misses are detected in two ways. The main method is the queue receive timeout. When the DDS calls xQueueReceive with a timeout set to the time remaining until the head task's deadline, a return of pdFALSE indicates that the timeout expired without a message arriving. That means the head task's deadline passed while it was still running. The DDS then removes it from the active list, sets its priority to LOW, records the overdue event, and adds the task to the overdue list.

There is also a sweep inside the handle_release function. Whenever a new task gets released, the DDS goes through the active list to check if any tasks have passed their deadlines already. This is needed for situations where multiple tasks get released at the same tick and a deadline expires in between processing the release messages, which would not trigger the queue timeout on its own. This came up in Test Bench 3 since all three tasks get released at the same tick every period.

In all the observed deadline misses across Test Benches 2 and 3, the DDS handled them correctly. The overdue task's priority was set to LOW to stop it from using more CPU time; it was moved to the overdue list, and the priorities of the remaining active tasks were adjusted so the next-earliest-deadline task could run. The system continued to run normally after each miss, without any cascading failures.

One thing to note is that overdue tasks are not re-released. When a periodic task misses its deadline, the current instance is added to the overdue list, but the next instance is still generated by the software timer at the next period. So a single deadline miss does not prevent future instances from being scheduled. But if the system is always overloaded, as in Test Bench 3, the same task will miss every period.

5.3 Discussion

The EDF implementation was verified by comparing the actual scheduling order with EDF theory for each test bench.

For Test Bench 1 (U = 0.823), all 8 task instances completed before their deadlines in the first hyper-period. At every scheduling decision, the task with the earliest absolute deadline ran. When deadlines were tied (like Task 1 and Task 2 at t = 0, both with a deadline of 500 ms), ties were broken by insertion order. EDF does not specify how to break ties, so any consistent method is valid.

For Test Bench 2 (U = 1.013), ten out of eleven task instances were completed on time. The one miss was on the sixth instance of Task 1, which is consistent with theory, since the utilization exceeds 1.0 by 1.3%. The tasks require 1519.5 ms of CPU time in total, but only 1500 ms are available, so no algorithm could have gotten all 11 done. EDF is the best you can do on a single processor, it pushes the deadline miss as far back as it can and gets the most tasks done before that happens.

For Test Bench 3 (U = 1.000), six of the nine task instances finished on time and three missed, one in each period. On a real system you cannot actually run at exactly 100% utilization because the scheduler itself takes some time to do its work. Task 3 was the one that missed every time because it always ended up running last in each period due to the tie-breaking order. Since it ran last it got hit with all the overhead that built up from the earlier tasks.

The sorting on the active list was working correctly throughout all the test benches. A task with a later deadline never ran before a task with an earlier one. insert_node kept the list in order by absolute deadline and adjust_priorities gave the task at the head PRIORITY_USER_HIGH while everything else got PRIORITY_USER_LOW.

6. Limitations and Possible Improvements

There are several limitations in the current implementation that could be improved in the future.

The biggest issue is the overhead from printf-based console output. Each call to sem_printf acquires the print mutex and blocks until the SWV console finishes transmitting. This is the main reason for the 1 ms deviations across all test benches and the cause of the deadline misses in Test Bench 3. In a real deployment, the print statements should either be removed entirely or replaced with a non-blocking ring buffer that a low-priority background task drains during idle time.

The system uses pvPortMalloc for allocating list nodes, but since heap_1.c does not support freeing memory, the completed and overdue lists grow without bound. For the three-task test benches running for one hyper-period, this is not a problem, but for systems with more tasks or longer run times, memory would eventually run out. Switching to heap_4.c would allow proper deallocation, but heap_4.c can cause heap fragmentation, which is risky on the STM32F4 with its limited RAM.

The Monitor Task runs at priority 4, which is above the user tasks, so it can report status even when user tasks are active. However, each time the monitor queries the DDS, it sends three messages to the DDS queue and waits for three replies, which adds overhead. Running the monitor less frequently helps reduce this.

The system does not currently measure or report processor utilization. This could be done by counting idle ticks in the FreeRTOS idle hook and comparing them to the total elapsed ticks. Another option would be to add timing fields to the dd_task struct to record when each execution segment starts and stops, and have the Monitor Task calculate utilization from that data.

7. Conclusion

This project built a Deadline-Driven Scheduler using the Earliest Deadline First algorithm by modifying the FreeRTOS priority-based scheduler and running it on the STM32F4 Discovery Board from its original fixed scheduling priority. The DDS system has five interface functions: release_dd_task, complete_dd_task, get_active_dd_task_list, get_completed_dd_task_list, and get_overdue_dd_task_list.

Our DDS system was tested on 3 different test benches; each had a higher CPU utilization. A higher utilization means the task periods are shorter and run more frequently within the given 1500ms hyperperiod. The first test bench utilization was below 100%, so the system ran smoothly with no missed deadlines.

This shows that EDF schedules tasks reliably when there is enough CPU time. Test bench 2 was to show that the monitor could accurately request the active, completed, and overdue lists from the monitor and print the counts of each to the screen periodically, with the counts displayed at the end of the hyper period (1500ms). We also increased utilization on this test bench to test what the system could do, as the execution time of all the tasks approached the CPU's time.

The third test bench was 100%, indicating that the CPU was fully utilized and that the system did not perform as expected. This was likely due to drift in execution time accumulated by printing to the screen, even though we reduced the amount we printed.
Only one generator task is used rather than having a separate one for each task. This cuts down on overhead and means there is only one task sending release messages to the DDS queue instead of three competing for it. The generator uses a software timer to know when to release tasks. For the list queries from the monitor, the DDS sends back pointers to the internal lists instead of making copies. Copying the whole list would take extra time and use more memory which is not worth it on the STM32F4.

The user tasks are all created once at startup with xTaskCreate and then reused for every period. Creating and deleting tasks over and over would risk fragmenting the heap since heap_1 does not free memory. The test benches only have three tasks running for one hyperperiod so this is not a huge deal right now, but if more tasks were added or the system had to run for multiple hyperperiods then pre-creating them would matter a lot more.

Our DDS performed well under low and near-100 % utilization, with scheduled tasks based on the closest absolute deadline; communication between tasks remained reliable via queues.

8 - Appendix

8.1 Source Code

For a more detailed explanation of code please see the **Software Design and Implementation** (Section 3) of this report.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdarg.h>
#include "stm32f4_discovery.h"
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "semphr.h"
#include "timers.h"

#define TEST_BENCH 3
#if TEST_BENCH == 1
#define T1_EXEC_MS 95
#define T1_PERIOD_MS 500
#define T2_EXEC_MS 150
#define T2_PERIOD_MS 500
#define T3_EXEC_MS 250
#define T3_PERIOD_MS 750

#elif TEST_BENCH == 2
#define T1_EXEC_MS 95
#define T1_PERIOD_MS 250
#define T2_EXEC_MS 150
#define T2_PERIOD_MS 500
#define T3_EXEC_MS 250
#define T3_PERIOD_MS 750

#elif TEST_BENCH == 3
#define T1_EXEC_MS 100
#define T1_PERIOD_MS 500
#define T2_EXEC_MS 200
#define T2_PERIOD_MS 500
#define T3_EXEC_MS 200
#define T3_PERIOD_MS 500

#else
#error "TEST_BENCH must be 1, 2, or 3"
#endif

#define PRIORITY_DDS 5
#define PRIORITY_MONITOR 4
#define PRIORITY_GENERATOR 3
#define PRIORITY_USER_HIGH 2
```

```
#define PRIORITY_USER_LOW 1

#define TICKS_TO_MS(t) ((uint32_t)((t) * 1000u / configTICK_RATE_HZ))
#define GENERATOR_WAKE_UP   ((uint32_t)1u)
typedef enum { PERIODIC, APERIODIC } task_type;

typedef struct {
   TaskHandle_t t_handle;
   task_type type;
   uint32_t task_id;
   uint32_t instance_id;
   uint32_t release_time;
   uint32_t absolute_deadline;
   uint32_t completion_time;
} dd_task;

typedef struct dd_task_node {
   dd_task task;
   struct dd_task_node *next;
} dd_task_list;

#define MSG_RELEASE 1u
#define MSG_COMPLETE 2u
#define MSG_GET_ACTIVE 3u
#define MSG_GET_COMPLETED 4u
#define MSG_GET_OVERDUE 5u

typedef struct {
   uint32_t msg_type;
   dd_task task;
   QueueHandle_t reply_queue;
} dds_message;

typedef struct { dd_task_list *list; } dds_reply;

static QueueHandle_t dds_queue;
static QueueHandle_t generator_queue;
static QueueHandle_t task1_release_queue;
static QueueHandle_t task2_release_queue;
static QueueHandle_t task3_release_queue;
static TimerHandle_t release_timer;
static SemaphoreHandle_t print_mutex;

static dd_task_list *active_list = NULL;
static dd_task_list *completed_list = NULL;
static dd_task_list *overdue_list = NULL;
static uint32_t next_instance_id = 1;
static TaskHandle_t task1_handle = NULL;
static TaskHandle_t task2_handle = NULL;
static TaskHandle_t task3_handle = NULL;

static TickType_t t1_next_release_tick = 0;
static TickType_t t2_next_release_tick = 0;
```

```
static TickType_t t3_next_release_tick = 0;
static TickType_t t1_next_deadline_tick = 0;
static TickType_t t2_next_deadline_tick = 0;
static TickType_t t3_next_deadline_tick = 0;

void DD_Scheduler_Task( void *pvParameters );
void DD_Generator_Task( void *pvParameters );
void Monitor_Task( void *pvParameters );
void User_Defined_Task_1( void *pvParameters );
void User_Defined_Task_2( void *pvParameters );
void User_Defined_Task_3( void *pvParameters );

void insert_node(dd_task_list **head, dd_task task);
dd_task_list *remove_node(dd_task_list **head, uint32_t instance_id);
uint32_t get_list_size(dd_task_list *head);
TickType_t get_next_timeout(void);
void adjust_priorities(void);
void handle_release(dds_message *msg);
void handle_complete(dds_message *msg);
void handle_deadline_miss(void);
uint32_t release_dd_task(TaskHandle_t h, task_type t, uint32_t id,
uint32_t relative_deadline);
void complete_dd_task(uint32_t instance_id);
dd_task_list *get_active_dd_task_list(void);
dd_task_list *get_completed_dd_task_list(void);
dd_task_list *get_overdue_dd_task_list(void);

static void dwt_init(void);
static void busy_wait_ms(uint32_t ms);
static QueueHandle_t get_release_queue(uint32_t id);
static TaskHandle_t get_task_handle_for(uint32_t id);
static TickType_t get_period_ticks(uint32_t id);
static TickType_t get_next_release_min(void);
static void arm_release_timer(void);
static void release_due_tasks(void);
static void release_one_task(uint32_t id, task_type type, TickType_t
rel, TickType_t dl);
static void release_timer_cb(TimerHandle_t xTimer);
static void sem_printf(const char *fmt, ...);

// prints stuff but uses mutex so threads dont mess up output  TBH just
safer printf
// kinda wraps vprintf and locks before printing then unlocks after
static void sem_printf(const char *fmt, ...)
{
   va_list args;
   if (print_mutex != NULL) { xSemaphoreTake(print_mutex,
portMAX_DELAY); }
   va_start(args, fmt);
   vprintf(fmt, args);
   va_end(args);
   if (print_mutex != NULL) { xSemaphoreGive(print_mutex); }
}
```

```
#define REG_DEMCR           (*((volatile uint32_t *)0xE000EDFCu))
#define REG_DWT_CTRL        (*((volatile uint32_t *)0xE0001000u))
#define REG_DWT_CYCCNT      (*((volatile uint32_t *)0xE0001004u))
#define DEMCR_TRCENA_BIT    (1UL << 24)
#define DWT_CYCCNTENA_BIT   (1UL << 0)

// init the cycle counter so we can measure time more accurately
// needed for busy wait otherwise timing would be off
static void dwt_init(void)
{
   REG_DEMCR |= DEMCR_TRCENA_BIT;
   REG_DWT_CYCCNT = 0;
   REG_DWT_CTRL |= DWT_CYCCNTENA_BIT;
}

// just wastes time for given ms using cpu cycles ,not ideal but ok here
// uses DWT counter so kinda precise  but still busy waiting
static void busy_wait_ms(uint32_t ms)
{
   uint32_t cycles_needed = ms * (SystemCoreClock / 1000UL);
   uint32_t cycles_done = 0;
   uint32_t prev_cyc = REG_DWT_CYCCNT;
   TickType_t prev_tick = xTaskGetTickCount();

   while (cycles_done < cycles_needed) {
       uint32_t now_cyc = REG_DWT_CYCCNT;
       TickType_t now_tick = xTaskGetTickCount();
       if (now_tick == prev_tick)
       {
           cycles_done += (now_cyc - prev_cyc);
       }
       prev_cyc = now_cyc;
       prev_tick = now_tick;
   }
}

// inserts task into linked list sorted by deadline
// earlier deadline = higher priority so goes closer to head
void insert_node(dd_task_list **head, dd_task task)
{
   dd_task_list *new_node;
   dd_task_list *curr;
   new_node = (dd_task_list *)pvPortMalloc(sizeof(dd_task_list)); //
alloc mem  for new node
   if (new_node == NULL) { return; }
   new_node->task = task;
   new_node->next = NULL;

   if (*head == NULL) { *head = new_node; return; }
   // insert at front if earlier deadline  ,EDF priority basically
   if (task.absolute_deadline < (*head)->task.absolute_deadline)
   {
```

```
        new_node->next = *head;
        *head = new_node;
        return;
    }

    curr = *head;
    // looping till correct spot  keeping list sorted by deadline
    while (curr->next != NULL &&
           curr->next->task.absolute_deadline <= task.absolute_deadline)
        curr = curr->next;

    new_node->next = curr->next;
    curr->next = new_node;
}

// removes a task from list using instance id
// returns the node removed so we can reuse it or move it to other list
dd_task_list *remove_node(dd_task_list **head, uint32_t instance_id)
{
    dd_task_list *prev, *curr;
    if (*head == NULL) return NULL;
    // special case: removing head node  gotta update head pointer
    if ((*head)->task.instance_id == instance_id)
    {
        dd_task_list *r = *head;
        *head = (*head)->next;
        r->next = NULL;
        return r;
    }

    prev = *head;
    curr = (*head)->next;
    while (curr != NULL) {
        // traverse list to find matching instance id
        if (curr->task.instance_id == instance_id)
        {
            prev->next = curr->next;
            curr->next = NULL;
            return curr;
        }
        prev = curr;
        curr = curr->next;
    }
    return NULL;
}

// counts how many nodes in a list  pretty straightforward
// used for monitor stats mostly
uint32_t get_list_size(dd_task_list *head)
{
    uint32_t n = 0;
    while (head != NULL) { n++; head = head->next; }
    return n;
```

```
}

// figures out how long scheduler should wait before next deadline miss
// if no tasks active then just wait forever basically
TickType_t get_next_timeout(void)
{
   TickType_t now, dl;
   // no tasks rn so scheduler can chill (wait forever)
   if (active_list == NULL) { return portMAX_DELAY; }
   now = xTaskGetTickCount();
   dl = (TickType_t)active_list->task.absolute_deadline;
   // deadline already passed so timeout immediately
   if (dl <= now) { return 0; }
   return dl - now;
}

// updates task priorities based on EDF
// first task (earliest deadline) gets high prio others low
void adjust_priorities(void)
{
   dd_task_list *curr = active_list;
   while (curr != NULL) {
       // updating priority dynamically based on EDF
       vTaskPrioritySet(curr->task.t_handle,
                        (curr == active_list) ? PRIORITY_USER_HIGH
                                              : PRIORITY_USER_LOW);
       curr = curr->next;
   }
}

// handles when a task is released into system
// sets release time + absolute deadline and adds to active list
void handle_release(dds_message *msg)
{
   TickType_t now = xTaskGetTickCount();
   msg->task.release_time = (uint32_t)now;
   // converting relative dl -> absolute dl
   msg->task.absolute_deadline = (uint32_t)(now +
(TickType_t)msg->task.absolute_deadline);
   insert_node(&active_list, msg->task); // add to active  list sorted
by deadline
   adjust_priorities();
   sem_printf("T%u R:%ums\n",
           (unsigned int)msg->task.task_id,
           (unsigned int)TICKS_TO_MS(msg->task.release_time));
}

// called when task finishes execution
// removes from active list and pushes into completed list
void handle_complete(dds_message *msg)
{
   // take task out of active list when done
```

```
   dd_task_list *node = remove_node(&active_list,
msg->task.instance_id);
   if (node != NULL)
   {
       node->task.completion_time = xTaskGetTickCount();
       vTaskPrioritySet(node->task.t_handle, PRIORITY_USER_LOW);
       node->next = completed_list; // push to completed list, not
sorted btw just stack
       completed_list = node;
       adjust_priorities();
       sem_printf("T%u C:%ums\n",
              (unsigned int)node->task.task_id,
              (unsigned int)TICKS_TO_MS(node->task.completion_time));
   } else {
       adjust_priorities();
   }
}

// if a task misses deadline we move it to overdue list
// also lower its priority so it doesnt block others
void handle_deadline_miss(void)
{
   dd_task_list *missed;
   if (active_list == NULL) { return; }
   missed = active_list; // take the earliest deadline task head as
missed
   active_list = active_list->next;
   missed->next = NULL;
   vTaskPrioritySet(missed->task.t_handle, PRIORITY_USER_LOW);
   missed->next = overdue_list;
   overdue_list = missed; // move it to overdue list
   adjust_priorities();
   sem_printf("T%u M:%ums\n",
           (unsigned int)missed->task.task_id,
           (unsigned int)TICKS_TO_MS(xTaskGetTickCount()));
}

// main scheduler task (core of DDS)
// waits for messages like release/complete and handles deadlines
void DD_Scheduler_Task( void *pvParameters )
{
   dds_message msg;

   for (;;) {
       TickType_t timeout = get_next_timeout();
       // wait for msg OR timeout
       // timeout = possible deadline miss
       BaseType_t received = xQueueReceive(dds_queue, &msg, timeout);

       if (received == pdTRUE)
       {
           switch (msg.msg_type) {
               case MSG_RELEASE:   handle_release(&msg);   break;
```

```
                case MSG_COMPLETE:  handle_complete(&msg);  break;
                case MSG_GET_ACTIVE:
                {
                    dds_reply r = { active_list };
                    xQueueSend(msg.reply_queue, &r, portMAX_DELAY);
                    break;
                }
                case MSG_GET_COMPLETED:
                {
                    dds_reply r = { completed_list };
                    xQueueSend(msg.reply_queue, &r, portMAX_DELAY);
                    break;
                }
                case MSG_GET_OVERDUE:
                {
                    dds_reply r = { overdue_list };
                    xQueueSend(msg.reply_queue, &r, portMAX_DELAY);
                    break;
                }
                default: break;
            }
        } else { handle_deadline_miss(); } // no msg = deadline missed
kinda clever tbh
    }
}

// sends a release msg to scheduler with task info
// also assigns unique instance id
uint32_t release_dd_task(TaskHandle_t h, task_type t, uint32_t id,
                     uint32_t relative_deadline)
{
    dds_message msg;
    uint32_t iid = next_instance_id++; // unique id for each task
instance, auto increment
    msg.msg_type = MSG_RELEASE;
    msg.task.t_handle = h;
    msg.task.type = t;
    msg.task.task_id = id;
    msg.task.instance_id = iid;
    msg.task.release_time = 0;
    msg.task.absolute_deadline = relative_deadline;
    msg.task.completion_time = 0;
    msg.reply_queue = NULL;
    // send msg to scheduler, blocking if needed
    xQueueSend(dds_queue, &msg, portMAX_DELAY);
    return iid;
}

// sends completion msg to scheduler when task is done
// pretty much signals DDS that work finished
void complete_dd_task(uint32_t instance_id)
{
    dds_message msg;
```

```
   msg.msg_type = MSG_COMPLETE;
   msg.task.instance_id = instance_id;
   msg.reply_queue = NULL;
   xQueueSend(dds_queue, &msg, portMAX_DELAY);
}

// asks scheduler for active tasks list
// uses queue to get reply (kind of request-response pattern)
dd_task_list *get_active_dd_task_list(void)
{
   // temp queue for reply
   QueueHandle_t rq = xQueueCreate(1, sizeof(dds_reply));
   dds_message msg; dds_reply reply;
   msg.msg_type = MSG_GET_ACTIVE; msg.reply_queue = rq;
   xQueueSend(dds_queue, &msg, portMAX_DELAY);
   xQueueReceive(rq, &reply, portMAX_DELAY);
   vQueueDelete(rq);
   return reply.list;
}

// same idea but for completed tasks
// used by monitor to print stats
dd_task_list *get_completed_dd_task_list(void)
{
   QueueHandle_t rq = xQueueCreate(1, sizeof(dds_reply));
   dds_message msg; dds_reply reply;
   msg.msg_type = MSG_GET_COMPLETED; msg.reply_queue = rq;
   xQueueSend(dds_queue, &msg, portMAX_DELAY);
   xQueueReceive(rq, &reply, portMAX_DELAY); // wait for scheduler
response
   vQueueDelete(rq);
   return reply.list;
}

// gets tasks that missed deadlines
// again just querying scheduler
dd_task_list *get_overdue_dd_task_list(void)
{
   QueueHandle_t rq = xQueueCreate(1, sizeof(dds_reply));
   dds_message msg; dds_reply reply;
   msg.msg_type = MSG_GET_OVERDUE; msg.reply_queue = rq;
   xQueueSend(dds_queue, &msg, portMAX_DELAY);
   xQueueReceive(rq, &reply, portMAX_DELAY);
   vQueueDelete(rq);
   return reply.list;
}

// runs periodically and prints stats, active/completed/overdue
void Monitor_Task( void *pvParameters )
{
   TickType_t last = xTaskGetTickCount();
   uint32_t an, cn, on;
   dd_task_list *a_list, *c_list, *o_list;
```

```
   for (;;) {
       // periodic monitor every ~1.5 sec
       vTaskDelayUntil(&last, pdMS_TO_TICKS(1500));

       a_list = get_active_dd_task_list();
       c_list = get_completed_dd_task_list();
       o_list = get_overdue_dd_task_list();

       an = get_list_size(a_list);
       cn = get_list_size(c_list);
       on = get_list_size(o_list);

       sem_printf("M:a-%u c-%u o-%u\r\n",
                  (unsigned int)an,
                  (unsigned int)cn,
                  (unsigned int)on);
   }
}

// actual task 1 workload
// waits for release signal then does busy work then completes
void User_Defined_Task_1( void *pvParameters )
{
   uint32_t iid;
   for (;;) {
       xQueueReceive(task1_release_queue, &iid, portMAX_DELAY);
       // wait until generator releases this task
       busy_wait_ms(T1_EXEC_MS);
       // simulate execution time
       complete_dd_task(iid);
   }
}

// same as task 1 but different execution time
// simulates another periodic task
void User_Defined_Task_2( void *pvParameters )
{
   uint32_t iid;
   for (;;) {
       xQueueReceive(task2_release_queue, &iid, portMAX_DELAY);
       busy_wait_ms(T2_EXEC_MS);
       complete_dd_task(iid);
   }
}

// third task  same pattern
// just different timing params
void User_Defined_Task_3( void *pvParameters )
{
   uint32_t iid;
   for (;;) {
       xQueueReceive(task3_release_queue, &iid, portMAX_DELAY);
```

```
        busy_wait_ms(T3_EXEC_MS);
        complete_dd_task(iid);
    }
}

// returns correct queue for each task id
// used when releasing tasks
static QueueHandle_t get_release_queue(uint32_t id)
{
    switch (id) {
        case 1u: return task1_release_queue;
        case 2u: return task2_release_queue;
        case 3u: return task3_release_queue;
        default: return NULL;
    }
}

// maps task id to its handle
// needed when sending release info to scheduler
static TaskHandle_t get_task_handle_for(uint32_t id)
{
    switch (id) {
        case 1u: return task1_handle;
        case 2u: return task2_handle;
        case 3u: return task3_handle;
        default: return NULL;
    }
}

// returns period in ticks for each task
// converts ms macros to RTOS ticks
static TickType_t get_period_ticks(uint32_t id)
{
    switch (id) {
        case 1u: return pdMS_TO_TICKS(T1_PERIOD_MS);
        case 2u: return pdMS_TO_TICKS(T2_PERIOD_MS);
        case 3u: return pdMS_TO_TICKS(T3_PERIOD_MS);
        default: return 0;
    }
}

// finds earliest next release among all tasks
// used to set timer correctly
static TickType_t get_next_release_min(void)
{
    TickType_t n = t1_next_release_tick;
    if (t2_next_release_tick < n) { n = t2_next_release_tick; }
    if (t3_next_release_tick < n) { n = t3_next_release_tick; }
    return n;
}

// releases a single task instance
// calculates relative deadline and sends it to DDS
```

```
static void release_one_task(uint32_t id, task_type type,
                            TickType_t rel, TickType_t dl)
{
   QueueHandle_t q = get_release_queue(id);
   TaskHandle_t h = get_task_handle_for(id);
   TickType_t relative_deadline = dl - rel; // compute relative deadline
from abs times
   uint32_t iid;

   if (q == NULL || h == NULL) { return; }

   iid = release_dd_task(h, type, id, (uint32_t)relative_deadline);
   xQueueSend(q, &iid, portMAX_DELAY);
}

// checks if any task should be released now (or overdue release)
// loops to catch up if multiple releases missed
static void release_due_tasks(void)
{
   TickType_t now = xTaskGetTickCount();
   while (t1_next_release_tick <= now) {
       // release tasks if we are behind schedule, catch up loop
       release_one_task(1u, PERIODIC, t1_next_release_tick,
t1_next_deadline_tick);
       t1_next_release_tick += get_period_ticks(1u); // move to next
release time
       t1_next_deadline_tick += get_period_ticks(1u);
   }
   while (t2_next_release_tick <= now) {
       release_one_task(2u, PERIODIC, t2_next_release_tick,
t2_next_deadline_tick);
       t2_next_release_tick += get_period_ticks(2u);
       t2_next_deadline_tick += get_period_ticks(2u);
   }
   while (t3_next_release_tick <= now) {
       release_one_task(3u, PERIODIC, t3_next_release_tick,
t3_next_deadline_tick);
       t3_next_release_tick += get_period_ticks(3u);
       t3_next_deadline_tick += get_period_ticks(3u);
   }
}

// sets timer for next release event
// basically schedules generator wakeup
static void arm_release_timer(void)
{
   TickType_t next = get_next_release_min();
   TickType_t now = xTaskGetTickCount();
   TickType_t delay = (next <= now) ? 1u : (next - now);
   // if already passed just trigger ASAP (1 tick)
   xTimerChangePeriod(release_timer, delay, 0);
}
```

```
// timer callback just sends it to generator task
// cant do heavy work here so just notify
static void release_timer_cb(TimerHandle_t xTimer)
{
   uint32_t tok = GENERATOR_WAKE_UP;
   (void)xTimer;
   // notify generator task (no blocking here)
   xQueueSend(generator_queue, &tok, 0);
}

// generates periodic tasks releases
// manages release timing using timer + queues
void DD_Generator_Task( void *pvParameters )
{
   TickType_t t0;
   uint32_t tok;

   // queue to send release signals to task1
   task1_release_queue = xQueueCreate(4, sizeof(uint32_t));
   task2_release_queue = xQueueCreate(4, sizeof(uint32_t));
   task3_release_queue = xQueueCreate(4, sizeof(uint32_t));

   // creating user task 1
   xTaskCreate(User_Defined_Task_1, "T1", configMINIMAL_STACK_SIZE +
128,
               NULL, PRIORITY_USER_LOW, &task1_handle);
   xTaskCreate(User_Defined_Task_2, "T2", configMINIMAL_STACK_SIZE +
128,
               NULL, PRIORITY_USER_LOW, &task2_handle);
   xTaskCreate(User_Defined_Task_3, "T3", configMINIMAL_STACK_SIZE +
128,
               NULL, PRIORITY_USER_LOW, &task3_handle);

   t0 = xTaskGetTickCount();
   t1_next_release_tick = t0;
   t1_next_deadline_tick = t0 + pdMS_TO_TICKS(T1_PERIOD_MS);
   t2_next_release_tick = t0;
   t2_next_deadline_tick = t0 + pdMS_TO_TICKS(T2_PERIOD_MS);
   t3_next_release_tick = t0;
   t3_next_deadline_tick = t0 + pdMS_TO_TICKS(T3_PERIOD_MS);

   release_timer = xTimerCreate("RelTmr", pdMS_TO_TICKS(1),
                                pdFALSE, (void *)0, release_timer_cb);
   release_due_tasks(); // release tasks at startup, so we dont wait
first timer
   arm_release_timer();

   for (;;) {
       xQueueReceive(generator_queue, &tok, portMAX_DELAY);
       release_due_tasks();
       arm_release_timer();
   }
}
```

```
// sets everything up like queues, tasks, scheduler
// then starts RTOS scheduler
int main(void)
{
   dwt_init();
   // main queue for scheduler comms
   dds_queue = xQueueCreate(16, sizeof(dds_message));
   generator_queue = xQueueCreate(8, sizeof(uint32_t));
   print_mutex = xSemaphoreCreateMutex();

   xTaskCreate(DD_Scheduler_Task, "DDS", configMINIMAL_STACK_SIZE, NULL,
PRIORITY_DDS, NULL);
   xTaskCreate(Monitor_Task, "Mon", configMINIMAL_STACK_SIZE, NULL,
PRIORITY_MONITOR, NULL);
   xTaskCreate(DD_Generator_Task, "Gen", configMINIMAL_STACK_SIZE, NULL,
PRIORITY_GENERATOR, NULL);
   // start RTOS  after this no coming back
   vTaskStartScheduler();
   while (1) {}
}

void vApplicationMallocFailedHook( void )
{
   // The malloc failed hook is enabled by setting
   // configUSE_MALLOC_FAILED_HOOK to 1 in FreeRTOSConfig.h.
   //
   // Called if a call to pvPortMalloc() fails because there is
insufficient
   // free memory available in the FreeRTOS heap.  pvPortMalloc() is
called
   // internally by FreeRTOS API functions that create tasks, queues,
software
   // timers, and semaphores.  The size of the FreeRTOS heap is set by
the
   // configTOTAL_HEAP_SIZE configuration constant in FreeRTOSConfig.h.
   for( ;; );
}

void vApplicationStackOverflowHook( xTaskHandle pxTask, signed char
*pcTaskName )
{
   ( void ) pcTaskName;
   ( void ) pxTask;
   // Run time stack overflow checking is performed if
configconfigCHECK_FOR_STACK_OVERFLOW is
   // defined to 1 or 2.  This hook
   // function is called if a stack overflow is detected.  pxCurrentTCB
can be
   // inspected in the debugger if the task name passed into this
function is corrupt.
   for( ;; );
}
```

```
void vApplicationIdleHook( void )
{
   volatile size_t xFreeStackSpace;

   // The idle task hook is enabled by setting configUSE_IDLE_HOOK to 1
in FreeRTOSConfig.h.
   //
   // This function is called on each cycle of the idle task.  In this
case it
   // does nothing useful, other than report the amount of FreeRTOS heap
that  remains unallocated.
   xFreeStackSpace = xPortGetFreeHeapSize();

   if( xFreeStackSpace > 100 )
   {
       //  By now, the kernel has allocated everything it is going to,
so
       //  if there is a lot of heap remaining unallocated then
       // the value of configTOTAL_HEAP_SIZE in FreeRTOSConfig.h can be
reduced accordingly.
   }
}
```

8.2 Test Bench #1 Console Output

SWV Console | SWV Data Trace | SWV Statistical Profiling

Port 0

```
T1 R:0ms
T2 R:0ms
T3 R:0ms
T1 C:95ms
T2 C:246ms
T3 C:496ms
T1 R:500ms
T2 R:500ms
T1 C:595ms
T2 C:745ms
T3 R:750ms
T1 R:1000ms
T2 R:1000ms
T3 C:1001ms
T1 C:1096ms
T2 C:1246ms
M:a-0 c-8 o-0
T1 R:1500ms
T2 R:1500ms
T3 R:1500ms
T1 C:1596ms
T2 C:1746ms
T3 C:1997ms
T1 R:2000ms
T2 R:2000ms
T1 C:2095ms
T2 C:2245ms
```

8.3 Test Bench #2 Console Output

SWV Console | SWV Data Trace | SWV Statistical Profiling

Port 0

```
T1 R:0ms
T2 R:0ms
T3 R:0ms
T1 C:95ms
T2 C:246ms
T1 R:250ms
T1 C:345ms
T1 R:500ms
T2 R:500ms
T3 C:592ms
T1 C:687ms
T1 R:750ms
T3 R:750ms
T2 C:838ms
T1 C:933ms
T1 R:1000ms
T2 R:1000ms
T1 C:1095ms
T1 R:1250ms
T3 C:1279ms
T2 C:1430ms
T1 M:1500ms
M:a-0 c-10 o-1
T1 R:1500ms
T2 R:1500ms
T3 R:1501ms
T1 C:1622ms
```

8.4 Test Bench #3 Console Output

SWV Console | SWV Data Trace | SWV Statistical Profiling

Port 0

```
T1 R:0ms
T2 R:0ms
T3 R:0ms
T1 C:100ms
T2 C:301ms
T3 M:500ms
T1 R:500ms
T2 R:500ms
T3 R:500ms
T1 C:601ms
T2 C:801ms
T3 M:1000ms
T1 R:1000ms
T2 R:1000ms
T3 R:1000ms
T1 C:1101ms
T2 C:1301ms
T3 M:1500ms
M:a-0 c-6 o-3
T1 R:1500ms
T2 R:1500ms
T3 R:1501ms
T1 C:1601ms
T2 C:1801ms
T1 R:2000ms
T2 R:2000ms
T3 R:2000ms
```